

# SparQ Attention: Bandwidth-Efficient LLM Inference

Luka Ribar<sup>\*1</sup> Ivan Chelombiev<sup>\*2</sup> Luke Hudlass-Galley<sup>\*1</sup> Charlie Blake<sup>1</sup> Carlo Luschi<sup>1</sup> Douglas Orr<sup>1</sup>

## Abstract

The computational difficulties of large language model (LLM) inference remain a significant obstacle to their widespread deployment. The need for many applications to support long input sequences and process them in large batches typically causes token-generation to be bottlenecked by data transfer. For this reason, we introduce **SparQ Attention**, a technique for increasing the inference throughput of LLMs by utilising memory bandwidth more efficiently within the attention layers, through selective fetching of the cached history. Our proposed technique can be applied directly to off-the-shelf LLMs during inference, without requiring any modification to the pre-training setup or additional fine-tuning. We show that SparQ Attention brings up to  $8\times$  savings in attention data transfers without substantial drops in accuracy, by evaluating Llama 2 and 3, Mistral, Gemma and Pythia models on a wide range of downstream tasks.

## 1. Introduction

Transformer models trained on large corpora of text have recently shown remarkable performance on complex natural language processing tasks (Achiam et al., 2023; Touvron et al., 2023). This has been attributed to the *in-context learning* capabilities that emerge with large-scale training, enabling arbitrary textual information (e.g. long instructions, chat histories, relevant documents) to be incorporated at inference-time (Wei et al., 2022).

To leverage the benefits of in-context learning, there has been demand for LLMs to support increasingly long input sequences. However, the standard inference optimisation

<sup>\*</sup>Equal contribution <sup>1</sup>Graphcore Research, United Kingdom  
<sup>2</sup>Synthesia, United Kingdom (work done while at Graphcore Research). Correspondence to: Luka Ribar, Luke Hudlass-Galley, Douglas Orr {luka, lukehg, douglaso}@graphcore.ai.

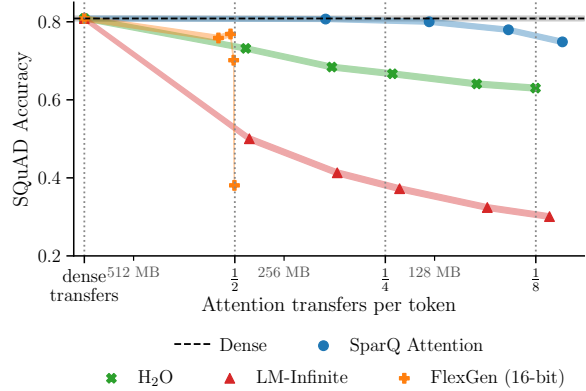


Figure 1: Llama 2 13B SQuAD 1-shot performance versus attention transfers over a range of compression ratios. SparQ Attention achieves matching performance, while transferring between 1/8 and 1/4 as much data as the original dense model. Line thickness shows  $\pm$  one standard error over 4000 examples (the uncertainty from a finite test set). This pattern is representative of the performance across various models and tasks, shown in Figures A1 to A3.

used to support in-context learning, *key-value (KV) caching* (Pope et al., 2023), is constrained by the need to fetch a large amount of data from memory when processing batches of long sequences. This in turn limits the speed at which tokens can be generated—a key usability metric for LLMs.

This bottleneck can be attributed to the auto-regressive nature of transformer generation. For each token generated, the full KV cache must be fetched from memory. The size of the KV cache scales linearly with the sequence length, as well as the batch size, thus rendering generation for long batched sequences increasingly memory bandwidth limited.

Despite this expensive cache-fetch at each step, tokens generally only attend to a small part of the sequence at a time (Vig, 2019; Yun et al., 2020). If it were possible to efficiently predict which tokens will have high attention scores, memory bandwidth efficiency could be significantly increased by only transferring the key-value pairs of high-scoring tokens.

Building upon this idea, we present **SparQ Attention**, a technique for significantly improving the memory bandwidth efficiency of transformer inference. By approximating attention scores using a subset of query and key components,

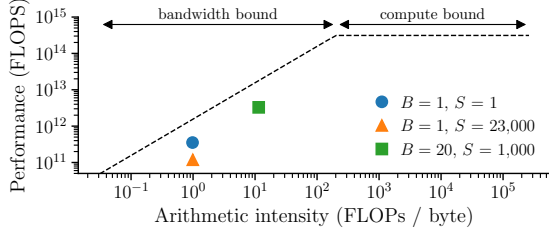


Figure 2: Roofline analysis of Llama 2 7B on A100 (40GB), highlighting that for a range of LLM inference settings with batch size  $B$  and sequence length  $S$ , practical performance is memory bandwidth bound.

we fetch only the most relevant tokens for each generation step, reducing the amount of data transferred without degrading the model.

We also provide a new set of challenging downstream task variants which we use to evaluate SparQ Attention. These are based on existing tasks, modified to assess a model’s ability to utilise information from long input sequences for multi-token generation. We show that SparQ Attention performs favourably compared to other state-of-the-art methods, giving up to  $8\times$  compression without substantial loss in accuracy. SparQ Attention is robust across tasks and models, demonstrated by evaluation on Llama 2 and 3, Mistral, Gemma and Pythia. We also provide benchmarks measured on IPU and GPU, showing the practical computational benefits of our approach.

## 2. Background

In this section we provide a straightforward framework to understand the computational efficiency of sequence generation using transformer models (similar to the modelling introduced by Kaplan et al. (2020)) and use it to motivate transfer-efficient attention mechanisms.

**Arithmetic intensity** Consider a compute unit capable of  $r_A$  scalar arithmetic operations per second that is connected to a memory via an interface which can transfer  $r_M$  scalar elements per second. Given a workload requiring  $\mathcal{A}$  arithmetic operations and  $\mathcal{M}$  transfers, and assuming concurrent compute and data transfer, the *arithmetic intensity* is defined as  $\mathcal{A}/\mathcal{M}$ . In LLM inference,  $\mathcal{A}$  is primarily a function of the size of matrix multiplications in the model, and  $\mathcal{M}$  depends on various factors such as the size of the KV cache, model size, and batch size. When the arithmetic intensity of the workload is less than the ratio  $r_A/r_M$ , execution time is limited by  $r_M$ , due to the data transfer taking longer than the computation in the concurrent setting.

The arithmetic intensity of typical sequence generation workloads in transformer models is shown in Figure 2, high-

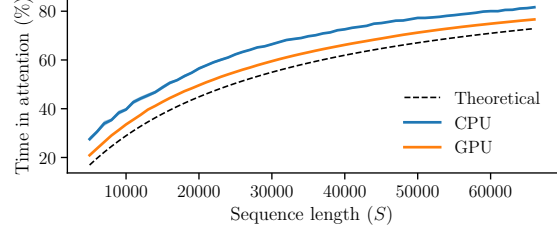


Figure 3: The proportion of time that is spent in attention layers during Llama 2 7B inference with a single sample when using `llama.cpp` on both CPU and GPU platforms. For more details, see Appendix D.

lighting that execution time is bandwidth bound, not compute bound. We provide a more general analysis of the arithmetic intensity of sequence generation in Appendix C, showing that it is typically bandwidth bound. A corollary of this is that the most effective way to accelerate transformer sequence generation is to reduce data transfers.

**Time in attention** Sequence generation with transformers is dominated by two types of computation. The first is a position-wise matrix multiplication between activations and parameters. The second is dot-product self-attention between activations (Vaswani et al., 2017). Assuming a standard transformer layer with model dimension  $d_m$ , batch size  $B$ , sequence length  $S$  and using Grouped Query Attention (GQA) (Ainslie et al., 2023) with  $g$  grouped-query heads per key-value head ( $g=1$  for standard multi-head attention), the proportion of data transfers associated with attention is given by

$$\frac{\mathcal{M}_{\text{attn}}}{\mathcal{M}_{\text{attn}} + \mathcal{M}_{\text{params}}} = \frac{\rho}{\rho + 6/B}, \quad (1)$$

where  $\mathcal{M}_{\text{params}}$  and  $\mathcal{M}_{\text{attn}}$  are the total data transfers of parameters and KV cache respectively, and  $\rho = S/(gd_m)$  is a variable we have introduced to capture the relevant model hyperparameters (see Appendix C). When  $\rho \gg 6/B$  (for example, with large  $S$  or  $B$ ), attention dominates data transfers, as the entire KV cache must be transferred during each generative step. This theoretical trend is backed up by empirical results from `llama.cpp` (Gerganov, 2024) benchmarks in Figure 3.

Since data transfer is the performance-limiting factor, and attention transfers dominate as sequence length is increased, there is a need for transfer-efficient alternatives to the standard attention mechanism.

## 3. Approximating Attention

In this section we examine several properties of the attention operation that enable us to introduce an accurate bandwidth-efficient approximation.

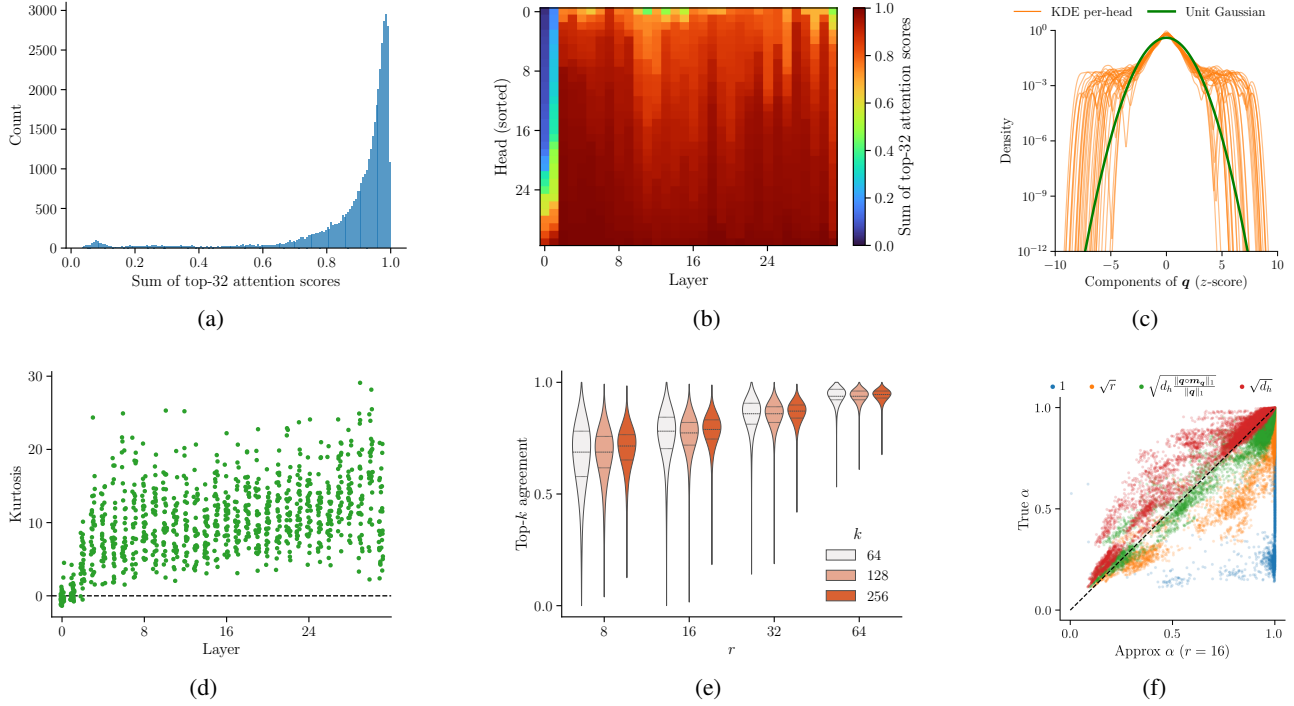


Figure 4: Statistics of Llama 2 7B, evaluated over 40 SQuAD queries, over all 32 layers  $\times$  32 heads unless noted. (a) Sum softmax output allocated to the 32 highest-scoring positions, demonstrating natural attention sparsity; (b) for each head. (c) Kernel density estimate (Rosenblatt, 1956) of components of  $\mathbf{q}$  in layer 16, showing heavy tails. (d) Fisher Kurtosis of  $\mathbf{q}$  components, for each head, showing that the query vector is leptokurtic for most heads. (e) Top- $k$  agreement between approximate and true scores for multiple values of  $r$  selected from query vector. Top- $k$  agreement is the proportion of the top- $k$  positions that are correctly predicted by an approximated softmax, using a projection of  $\mathbf{q}$ . (f) Agreement between the coverage  $\alpha$  based on estimated scores versus the true mass of the top 128 scores, for different softmax temperatures (a point for each example  $\times$  head), showing the importance of correct temperature. Further analysis is presented in Appendix E.

Consider a single attention *query head* with the head dimension  $d_h$ , processing an input token sequence of length  $S$ . During autoregressive generation, the output of the attention head is calculated as:

$$\mathbf{y} = \text{softmax}\left(\frac{\mathbf{q} \cdot \mathbf{K}^\top}{\sqrt{d_h}}\right) \cdot \mathbf{V} \quad (2)$$

where  $\mathbf{q}$  is the query, and  $\mathbf{K} \in \mathbb{R}^{S \times d_h}$  and  $\mathbf{V} \in \mathbb{R}^{S \times d_h}$  are the key and value caches respectively. When using GQA (Ainslie et al., 2023),  $\mathbf{K}$  and  $\mathbf{V}$  are shared across  $g$  query heads.

For each forward pass, we need to fetch the key and value matrices from memory, as well as write (append)  $\mathbf{k}$  and  $\mathbf{v}$  vectors for the current token, giving a total number of elements transferred per attention head:

$$\mathcal{M}_{\text{dense}} = 2 S d_h + 2 d_h \quad (3)$$

where the first term corresponds to reading the  $\mathbf{K}$  and  $\mathbf{V}$  caches and the second term corresponds to writing the cur-

rent  $\mathbf{k}$  and  $\mathbf{v}$  to memory. Memory transfer may be equivalently expressed in terms of bytes, however we use scalar elements to disentangle cache compression methods from the number format used to represent the cache.

**Attention scores sparsity** First, consider the *attention scores*  $\mathbf{s} \in (0, 1)^S$  in Equation (2):

$$\mathbf{s} = \text{softmax}\left(\frac{\mathbf{q} \cdot \mathbf{K}^\top}{\sqrt{d_h}}\right) \quad (4)$$

Due to the normalising effect of the softmax function, the resulting  $\mathbf{s}$  vector is *sparse* (see Figures 4a and 4b), i.e. we can find a boolean mask  $\mathbf{m}_s \in \{0, 1\}^S$  corresponding to the top- $k$  elements in  $\mathbf{s}$  ( $k \ll S$ ) such that:

$$\mathbf{y}_1 = (\mathbf{s} \circ \mathbf{m}_s) \cdot \mathbf{V} \approx \mathbf{s} \cdot \mathbf{V} \quad (5)$$

As a result, only the values  $\mathbf{v}_i$  corresponding to the non-zero elements of  $\mathbf{m}_s$  need to be fetched from memory. However, the algorithm still requires fetching the full  $\mathbf{K}$  from memory in order to calculate the attention scores  $\mathbf{s}$ , limiting the minimum amount of data transferred to  $\frac{1}{2} \mathcal{M}_{\text{dense}}$ .